WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Sophia Munoz

Email: mailto:munozsa@mail.uc.edu

Figure 1: Sophia's headshot

Lab 2 - Front-end Web Development

The Lab's Overview

For Lab2 there were two tasks. Task 1 has two parts. Part `a` was creating webpages with the basic HTML tags. The webpage implemented used basic HTML tags with forms and an adjusted image. For Part `b`, I used simple JavaScript code within the HTML file to implement and show the Date and Email when clicked by the user. And a digit and analog clock was also integrated into the webpage.

Task 2 has four parts. Part `a` was implementing Ajax GET requests and keypress logging. Part `b` was implementing CSS with a class in JavaScript. There are three types of CSS implementation (inline, internal, external). For Part `c`, I implemented `jQuery` Ajax GET and POST requests to `echo.php` using the existing class for the button and adding functions for each request. For Part `d`, I integrated Web APIs with Ajax and fetch().

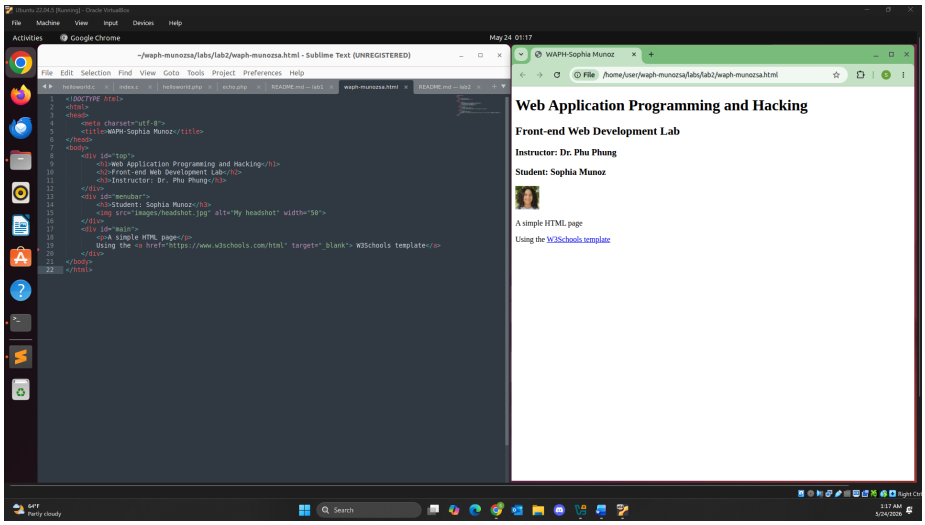
Outcomes I learned from this lab were a great amount of Front-End Web Development that will definitely be useful for Project 1. I also learned about how integrating Ajax asynchronously supports better performance in data handling.

Lab2 Folder: https://github.com/munozsophia/waph-munozsa/tree/main/labs/lab2.

Task 1: Basic HTML with forms, and JavaScript

a. HTML I developed the `waph-munozsa.html` file with basic tags, my headshot, and two forms for user input. For this lab I created a new images folder within the Lab 2 folder so that the webpage can access my `headshot.jpg` file.

To format the headshot within the webpage to 50 pixels, I entered `<img src="images/headshot.jpg" alt="My headshot" width="50">`.



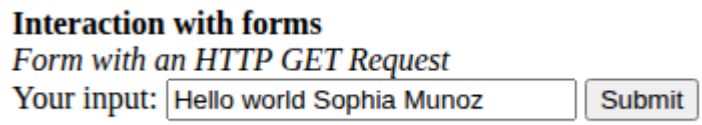
HTML Page with Headshot Image

To add the user input forms to the webpage, I used the <form> tag and <input> element with the code below to handle and display the user input to the `echo.php` web application.

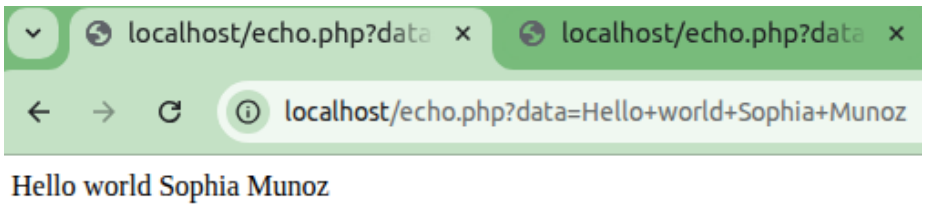
```
<b>Interaction with forms</b>
<div>
   <i>Form with an HTTP GET Request</i>
   <form action="/echo.php" method="GET">
      Your input: <input name="data">
      <input type="submit" name="Submit">
   </form>
</div>
<div>
```

```
   <i>Form with an HTTP POST Request</i>
   <form action="/echo.php" method="POST">
      Your input: <input name="data">
      <input type="submit" name="Submit">
   </form>
</div>
```

Below is an input from the user (using the GET method)

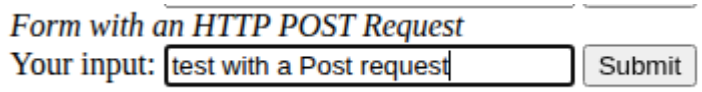


HTTP GET Request Input

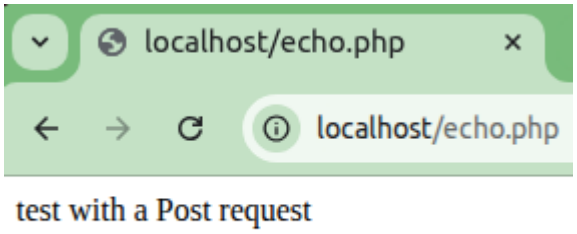


HTTP GET Request Output

Below is an input from the user (using the POST method)



HTTP POST Request Input

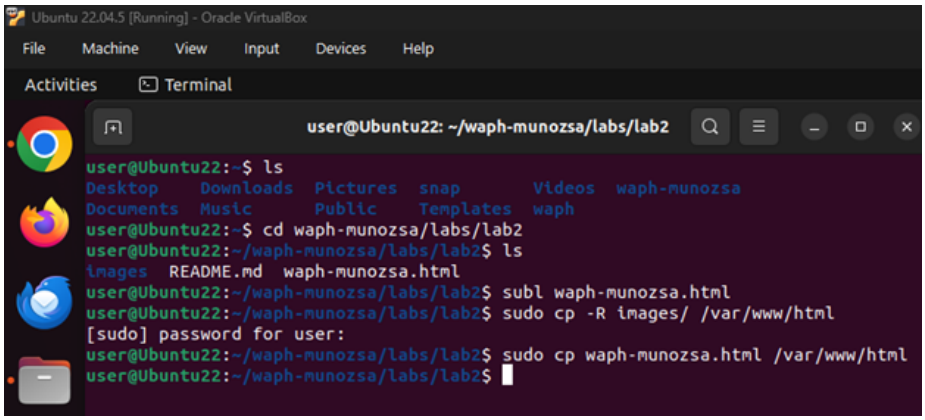


HTTP POST Request Output

As you can see from the images above, the data can be seen in the URL for the HTTP GET Request unlike the HTTP POST Request where the data is in the HTTP Headers.

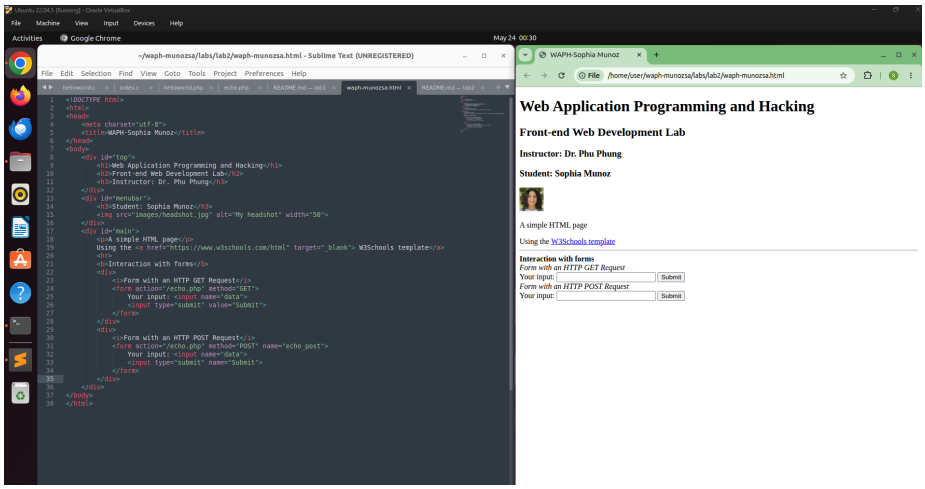
To view this webpage I ran:

- `$ sudo cp waph-munozsa.html /var/www/html` to deploy HTML page to web server root
- `$ sudo cp -R images/ /var/www/html` to deploy image to web server root



HTML Deployment Commands in Terminal

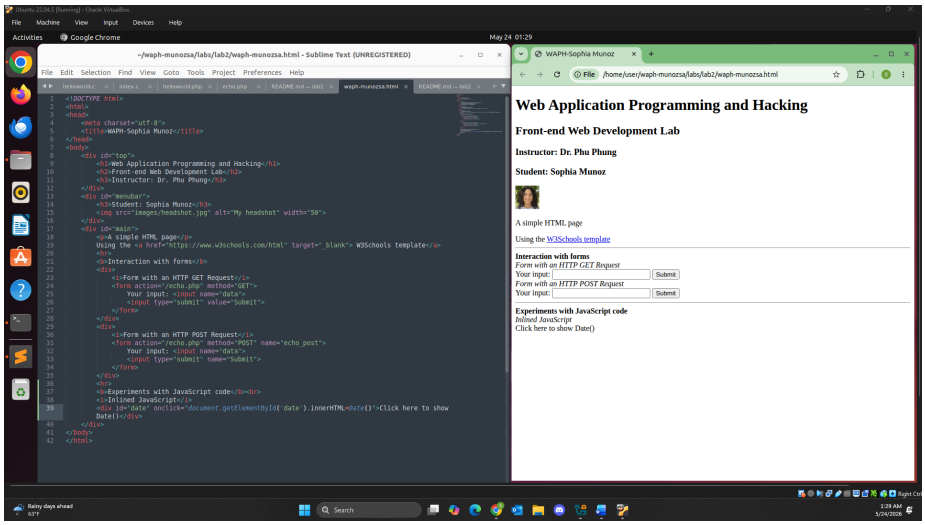
I then entered `localhost/waph-munozsa.html` in the browser. With the webpage rendered this what it looks like with my headshot and the form.



HTML Headshot and Form Webpage

b. Simple JavaScript Write the JavaScript code in your HTML page:

- The following is the inline JavaScript code for displaying time and keypress.



HTML JavaScript Inline Code Displaying Time

Below is the inline JavaScript code that displays the time and date when it is clicked by the user.

```
<b>Experiments with JavaScript code</b><br>
<i>Inlined JavaScript</i>
<div id="date"
     onclick="document.getElementById('date')
     .innerHTML=Date()">Click here to show Date()
</div>
```

Experiments with JavaScript code
Inlined JavaScript
Click here to show Date()

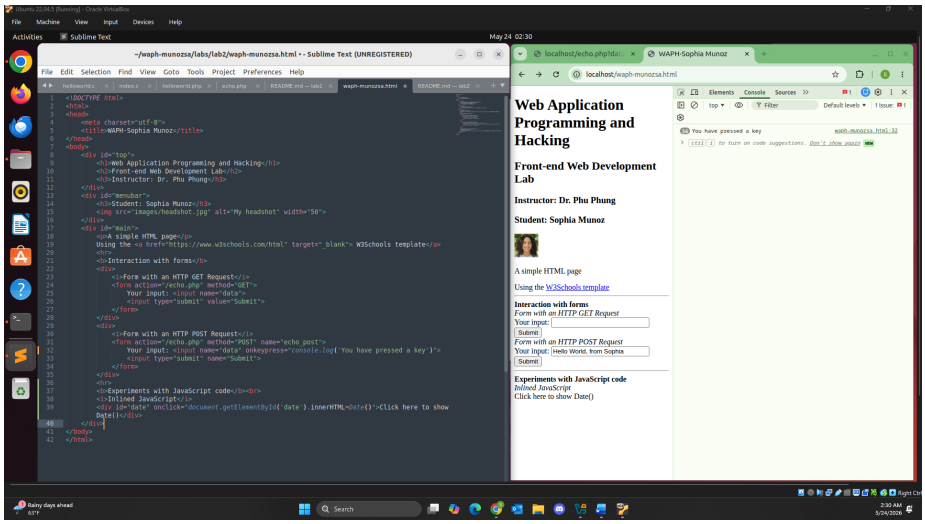
HTML JavaScript Before Click

Experiments with JavaScript code
Inlined JavaScript
Sun May 24 2026 01:29:39 GMT-0400 (Eastern Daylight Time)

HTML JavaScript After Click

Below is the inline JavaScript code to log when a key is pressed in the POST method.

```
<form action="/echo.php" method="POST" name="echo_post">
   Your input: <input name="data" onkeypress="console.log('You have pressed a key')">
   <input type="submit" name="Submit">
</form>
```



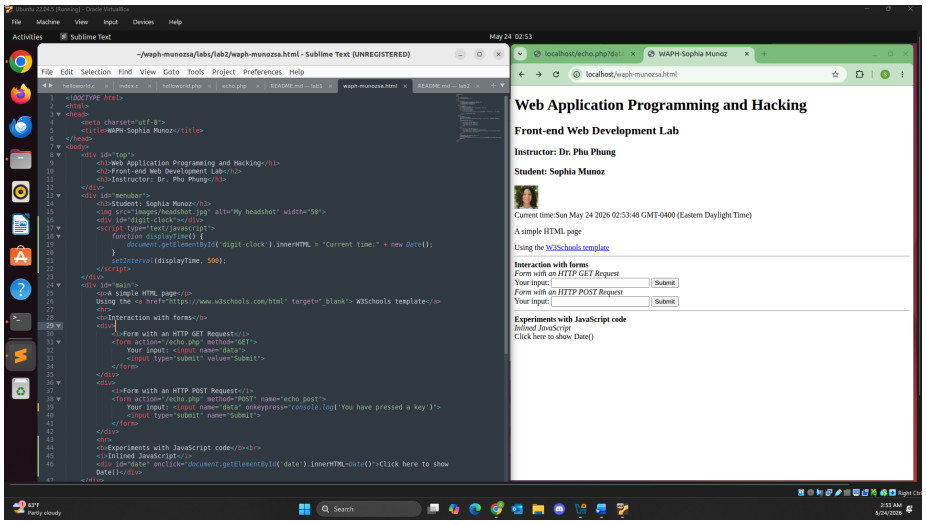
HTML JavaScript on keypress

- Below I used the <script> tag to display the digit clock below my headshot.

```
<div id="digit-clock"></div>
<script type="text/javascript">
   function displayTime() {
      document.getElementById('digit-clock').innerHTML = "Current time:" + new Date();
   }
   setInterval(displayTime, 500);
</script>
```

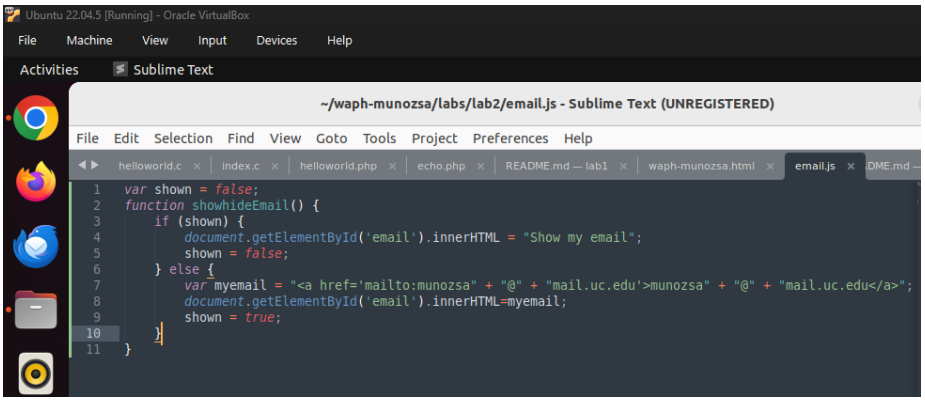
Current time:Sun May 24 2026 02:57:24 GMT-0400 (Eastern Daylight Time)

HTML JavaScript Digit Clock a Closer Look

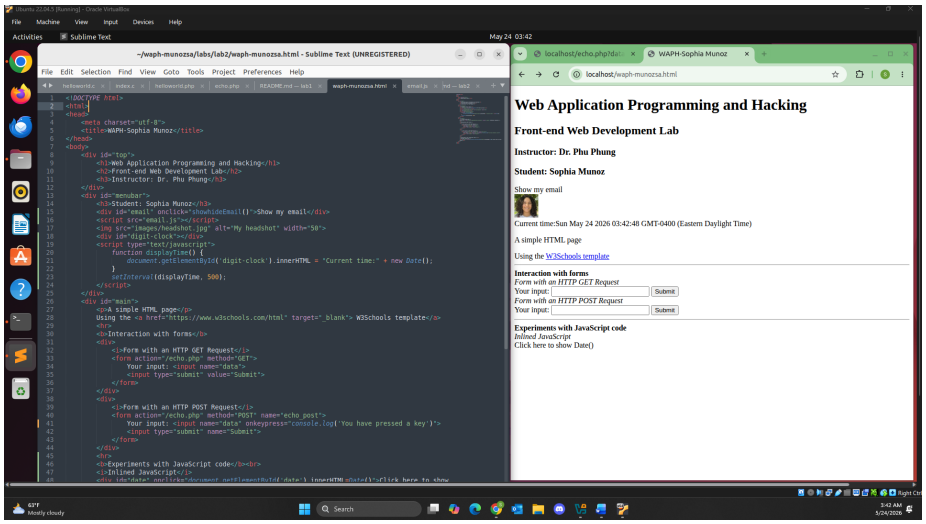


HTML JavaScript Digit Clock

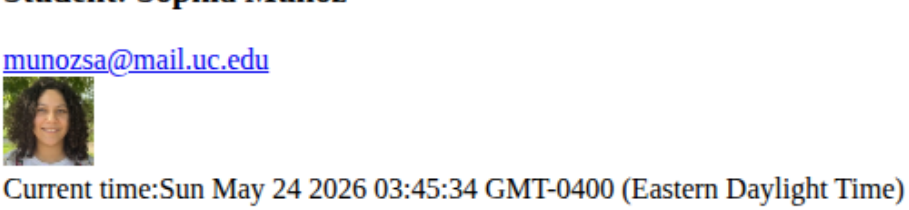
- Below I used the code implement the email and it appearing once clicked by the user.



HTML JavaScript email.js Code



HTML JavaScript Email Webpage



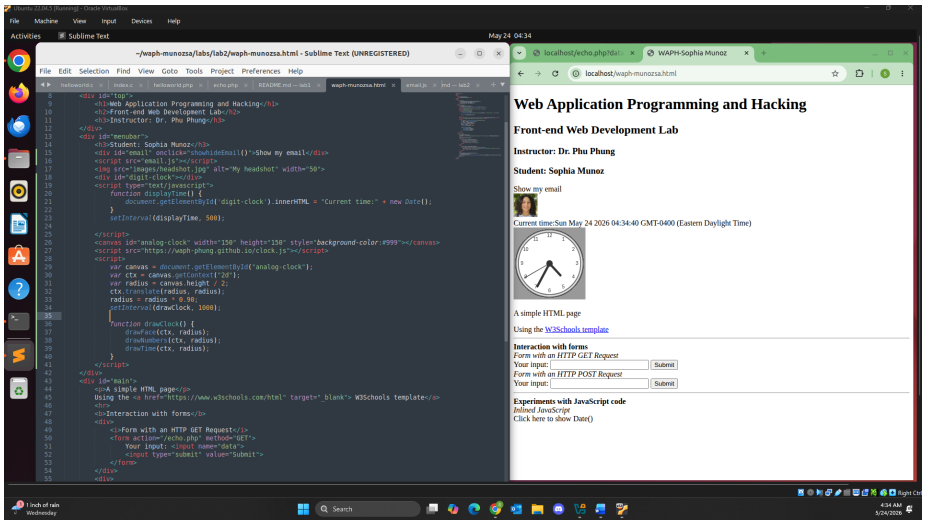
HTML JavaScript Email Clicked

- Below I implemented the code below from an external source of JavaScript code to display an analog clock.

```
<canvas id="analog-clock" width="150" height="150" style="background-color:#999"></canvas>
<script src="https://waph-phung.github.io/clock.js"></script>
<script>
   var canvas = document.getElementById("analog-clock");
   var ctx = canvas.getContext("2d");
   var radius = canvas.height / 2;
   ctx.translate(radius, radius);
   radius = radius * 0.90;
   setInterval(drawClock, 1000);

   function drawClock() {
     drawFace(ctx, radius);
     drawNumbers(ctx, radius);
     drawTime(ctx, radius);
   }
```

```
</script>
```



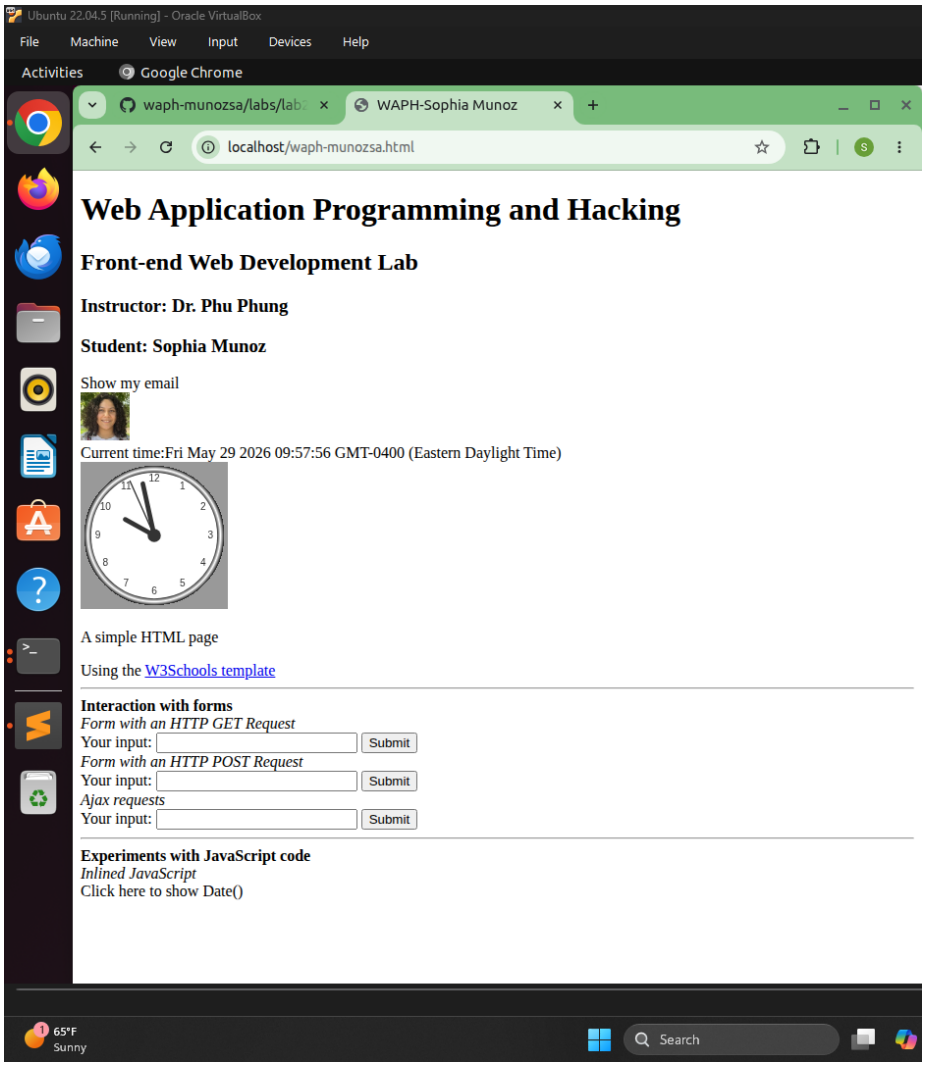
HTML JavaScript Analog Clock

Task 2: Ajax, CSS, jQuery, and Web API integration

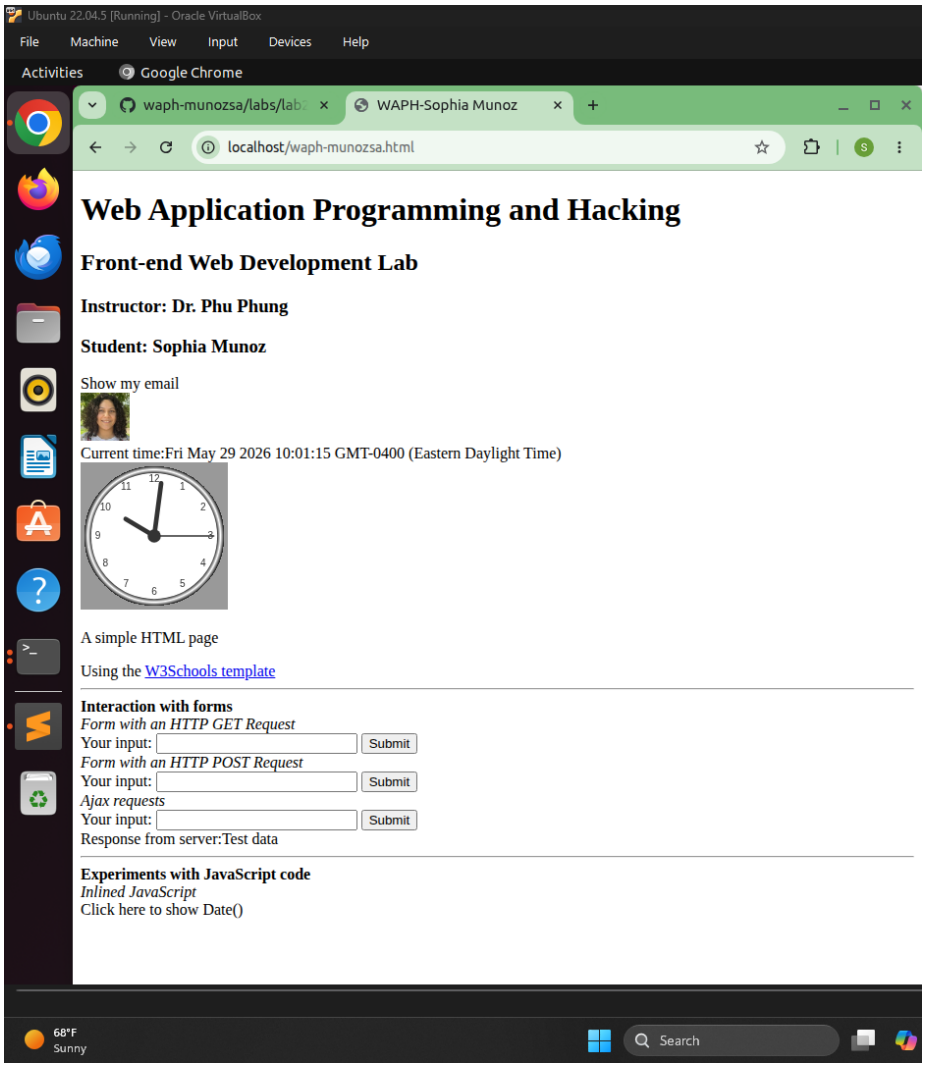
a. Ajax I implemented the JavaScript code below for the Ajax GET request. This allows for the user input to be seen as a key is pressed. And as you can see from the final image for the HTTP requests/responses, that the request URL contains the data input by the user (`http://localhost/echo.php?data=Another%20test%20data`) and that the status is `200 OK`. Implementing Ajax asynchronously allows for data handling to be dealt with better performance then when it is synchronous.

```
<script>
   function getEcho() {
      var input = document.getElementById("data").value;
      if (input.length == 0) {
         return;
      }
      var xhttp = new XMLHttpRequest();
      xhttp.onreadystatechange = function() {
         if (this.readyState == 4 && this.status == 200) {
            console.log("Received data=" + xhttp.responseText);
            document.getElementById("response")
                    .innerText= "Response from server:" + xhttp.responseText;
            // code to show the data
         }
      }
      xhttp.open("GET", "echo.php?data=" + input, true);
```

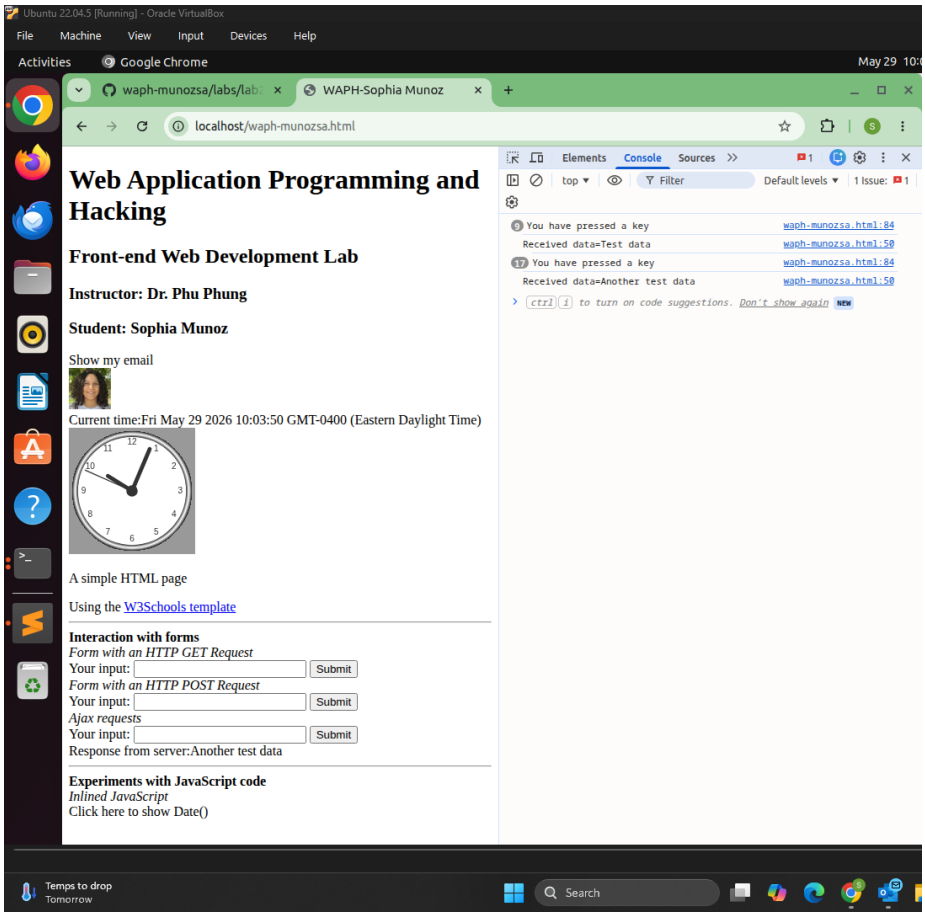
```
        // code to create an Ajax request
        xhttp.send(); // code to send the request
        document.getElementById("data").value="";
    }
</script>
```



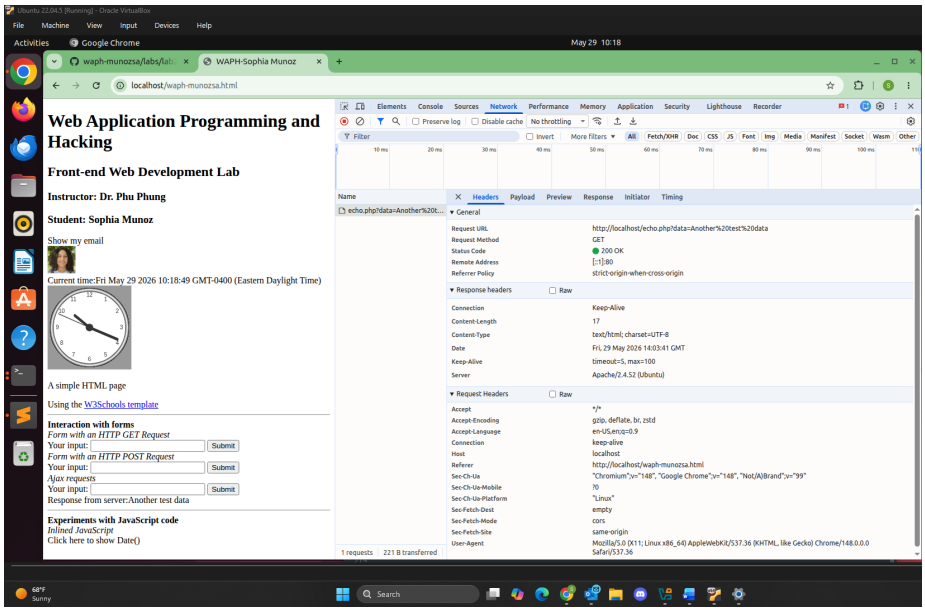
Ajax JavaScript GET Request Button Implemented



Ajax JavaScript Server Response



Ajax JavaScript keypress



Ajax JavaScript Network Outcome

b. CSS The inline CSS implementation of JavaScript code is below. While it wasn't implemented into my `waph-munozsa.html`, an example is provided.

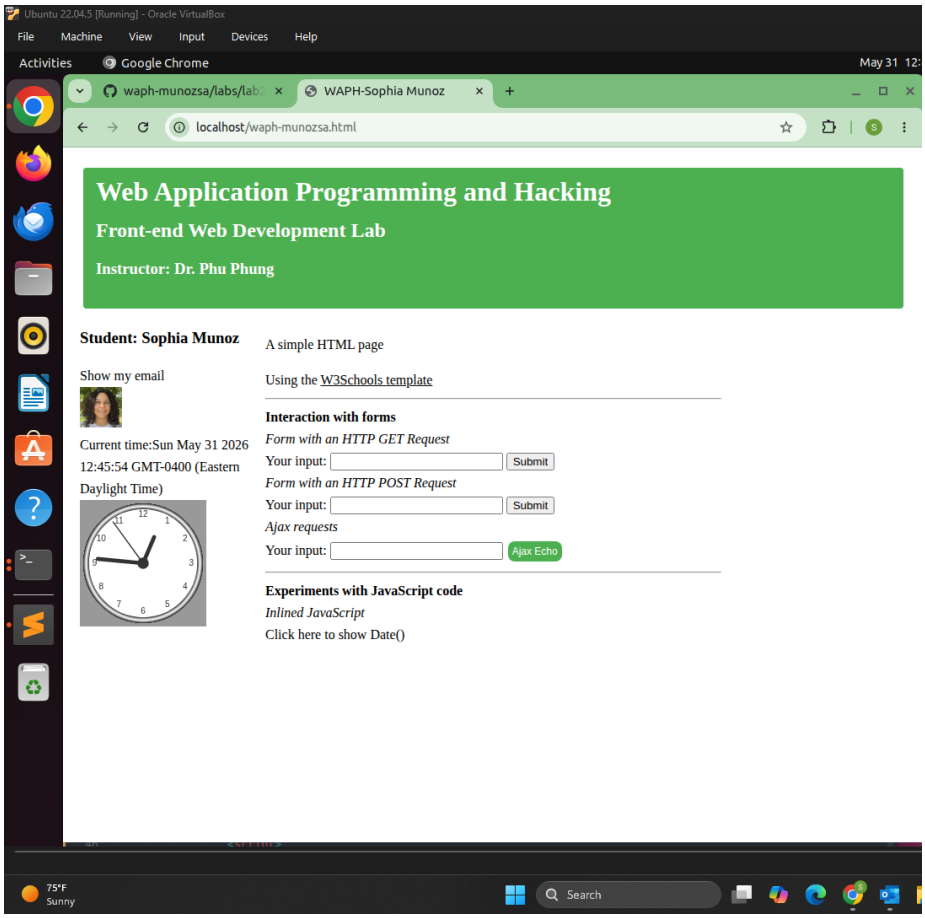
```
<h1 style="color: blue;">Blue Heading</h1>
```

The external CSS implementation of JavaScript code is below. The Remote CSS was implemented between the `<head></head>` tags of `waph-munozsa.html`. To complete this implementation I had to reorganize the webpage so it was a bit more neat. I also added a `container wrapper` class to wrap around the `menubar` and `main` tags. I used the `<script></script>` tag to encapsulate all the functions that have been implemented so far.

```
<link rel="stylesheet"
      type="text/css"
      href="https://waph-uc.github.io/style1.css">
```

The internal CSS implementation of JavaScript code is below, including the implementation of the class button round.

```
<input class="button round" type="button" value="Ajax Echo" onclick="getEcho()">
```

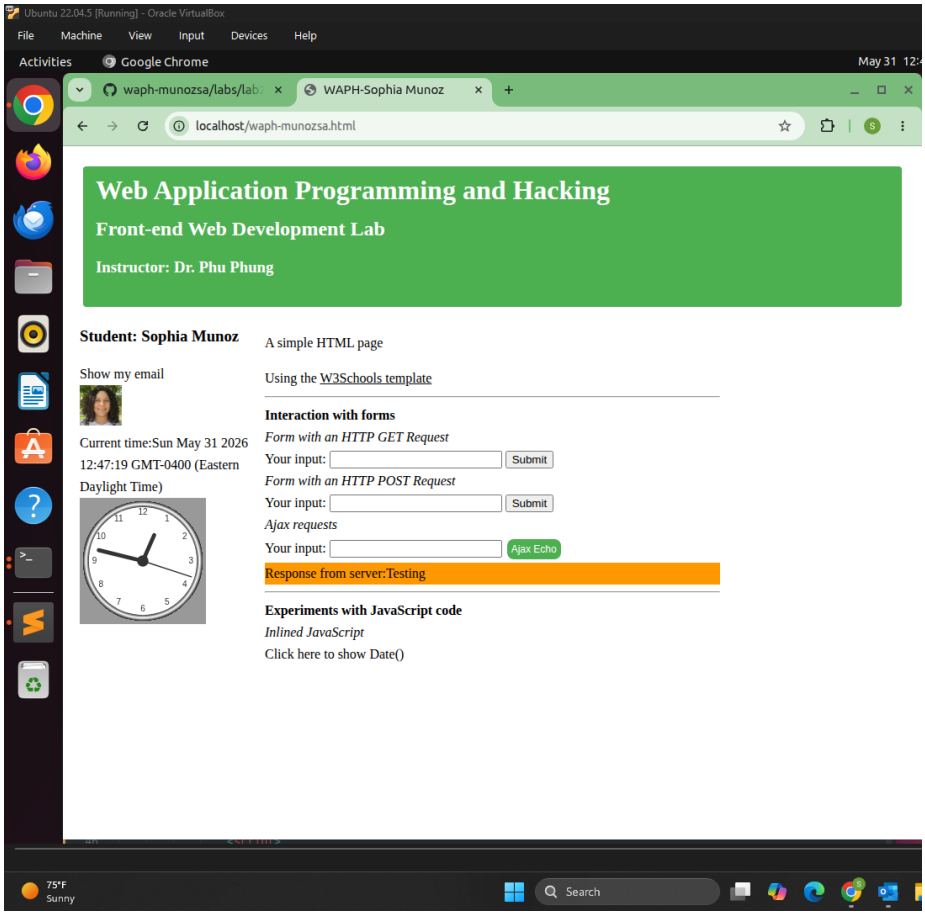


CSS JavaScript Button Implemented

Below is the button round class implementation.

```
<style>
   .button {
      background-color: #4CAF50; /* Green */
      border: none;
      color: white;
      padding: 5px;
      text-align: center;
      text-decoration: none;
      display: inline-block;
      font-size: 12px;
      margin: 4px 2px;
      cursor: pointer;
   }
   .round {border-radius: 8px;}
```

```
   #response {background-color: #ff9800;} /* Orange */
</style>
```



CSS JavaScript Server Response

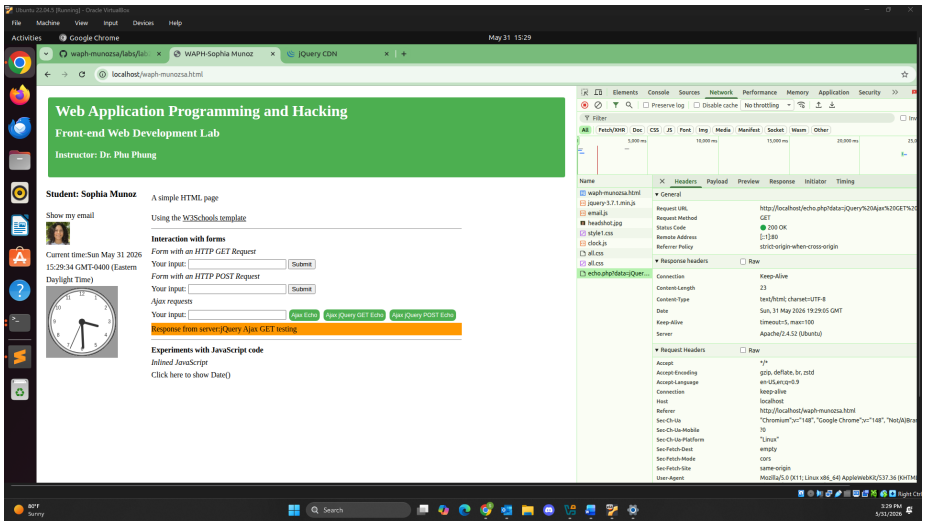
c. jQuery I added the jQuery libary with the code below.

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
        integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs0OEvtFFmqPF/lYI/Cxo="
        crossorigin="anonymous"></script>
```

i. An Ajax GET request to `echo.php` web application occurs when the `Ajax jQuery GET Echo` button is clicked and the server responds. From the image below, the GET method can be seen from the Inspect page. The code implemenation is below.

```
<script>
   function jQueryAjax() {
      var input = $("#data").val();
```

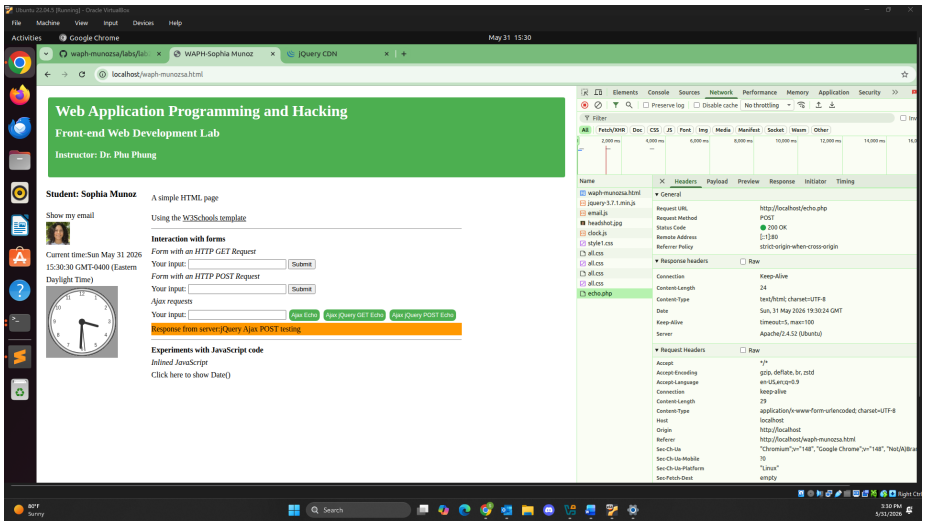
```
        if (input.length == 0) return;
        $.get("echo.php?data=" + input,
            function(result) {
                $("#response").html("Response from server:" + result);
            }
        );
        $("#data").val("");
    }
</script>
```



jQuery Ajax GET Request

ii. An Ajax POST request to `echo.php` web application occurs when the `Ajax jQuery POST Echo` button is clicked and the server responds. From the image below, the POST method can be seen from the Inspect page. The code implementation is below.

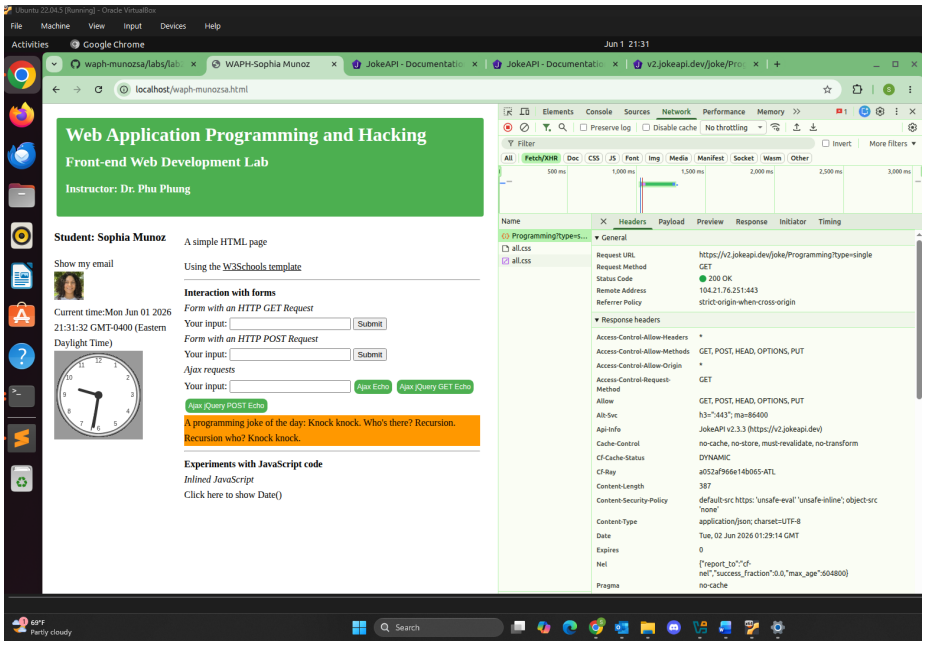
```
<script>
    function jQueryAjaxPost() {
        var input = $("#data").val();
        if (input.length == 0) return;
        $.post("echo.php",{data: input},
            function(result) {
                $("#response").html("Response from server:" + result);
            }
        );
        $("#data").val("");
    }
</script>
```



jQuery Ajax POST Request

d. Web API integration **i.** I used Ajax on https://v2.jokeapi.dev/joke/Programming?type=single, with the code below I integrated a joke simulator with jQuery Ajax and no input. As you can see in the image below the request was of type `XHR`. Anytime the page is reloaded, a new joke (related to programming) is generated.

```
<script>
   $.get("https://v2.jokeapi.dev/joke/Programming?type=single",
      function(result) {
         console.log("From jokeAPI: " + JSON.stringify(result));
         $("#response").html("A programming joke of the day: " + result.joke);
      }
   ) // this will be executed automatically
</script>
```



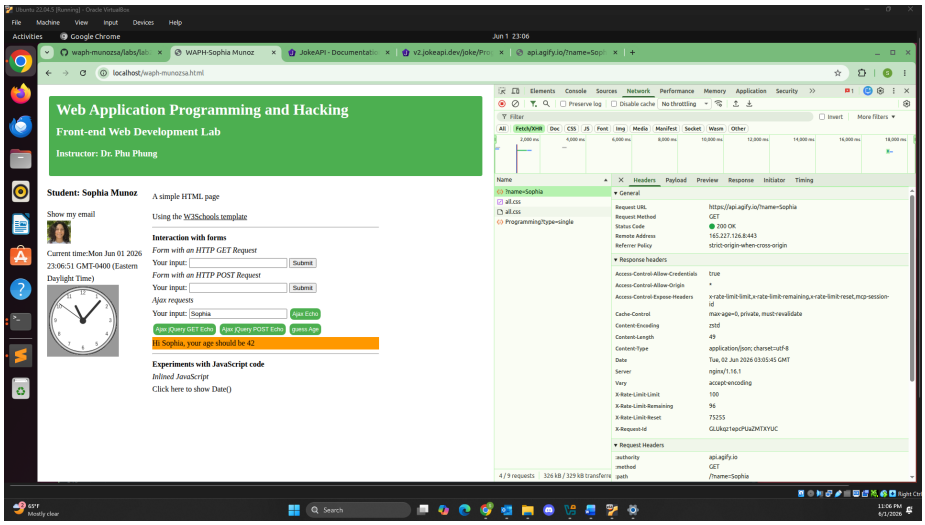
Web API Ajax Integration

ii. I used the `fetch` API on https://api.agify.io/?name=input, with the code below I integrated an age guesser based on a name inputted by the user with fetch(). As you can see in the image below the request was of type `fetch`.

```
<script>
      async function guessAge(name) {
      const response = await fetch("https://api.agify.io/?name=" + name);
      const result = await response.json();
      $("#response").html("Hi " + name + ", your age should be " + result.age);
   }
</script>
```

The code below implemented the new button with the existing Ajax requests form.

```
<input class="button round"
       type="button"
       value="guess Age"
       onclick="guessAge($('#data').val())">
```



Web API fetch() Integration